

Microduck a návrh Duckbotu

Co je na Microducku technicky zajímavé

Podle oficiálních materiálů Pollen Robotics je Microduck přibližně 25 cm vysoký robot s hmotností okolo 800 g. Používá 15 servomotorů, kameru, LiDAR, dvě IMU a řídicí smyčku s frekvencí 50 Hz. Na palubě běží Rockchip RK3566 a pohybové politiky se načítají jako ONNX modely.

Nejde tedy o jeden program v Arduinu. Systém má několik vrstev:

```
hlas, ovladač, kamera nebo aplikace
|
příkaz / záměr
|
řídící počítač robota
stav, politika, bezpečnost
|
realtime řízení serv
|
serva + IMU + další senzory
```

Oficiální repozitář je dobrý hlavně jako studijní materiál pro architekturu. Motorovou sběrnici a bezpečnost vlastní jedna řídicí služba, ostatní části jí posílají záměry typu „jdi dopředu“ nebo „sedni si“, nikoli přímé zápisy do motorů. To je důležitá vlastnost, kterou bychom měli převzít i pro Duckbot.

Co umí hned

Oficiální Microduck uvádí tyto připravené pohyby:

- chůze podle rychlosti a směru,
- sednutí a postavení,
- kopnutí do míče,
- sebrání jednoduchého předmětu zobákem,
- jízda na kolečkách,
- postavení se po pádu,
- zvukové reakce a kvákání.

To neznamena, že každý nový robot tyto schopnosti automaticky získá. Jsou výsledkem konkrétní mechaniky, kalibrace, senzorů a natrénovaných politik. U vlastního Duckbotu musíme každou schopnost nejdřív naprogramovat nebo natrénovat.

K čemu je Dev Pack

Dev Pack není druhý robot ani balík nových funkcí. Je to sada náhradních dílů a pomůcek pro člověka, který chce robota rozebírat, opravovat a vyvíjet pro něj vlastní chování.

Podle nabídky obsahuje:

- **3 náhradní motory** pro opravy, testování a experimenty s jiným kusem,
- **5 motorových kabelů** pro náhradu poškozeného kabelu nebo pokusy s novým zapojením,
- **2 baterie** pro delší práci a rychlé střídání vybitých akumulátorů,
- **dvojitou nabíječku** pro nabíjení obou baterií současně,
- **10 NFC tagů** pro vlastní interakce, spouštění akcí a chování podle NFC,
- **Hugging Face kredit** určený na cloudové úlohy, například trénování robota,
- **šroubovák a sadu náhradních šroubků** pro servis a montáž.

Pro vývoj je nejdůležitější kombinace náhradních motorů, kabelů, baterií a Hugging Face kreditu. U chodícího robota je motor spotřební riziková součást: špatná kalibrace, pád nebo zablokovaný kloub ho může přetížit. Dev Pack proto snižuje dobu mimo provoz a umožňuje bezpečnější experimentování.

Co Dev Pack **neobsahuje**:

- nový procesor nebo rychlejší řídicí počítač,
- další senzory jako kameru, LiDAR nebo IMU,
- mechanickou přestavbu robota,
- automatický nástroj na učení nových schopností.

NFC tagy samy o sobě robota nic nenaučí. Jsou to identifikátory, které může software použít jako spouštěč: například tag „tanec“ vybere uloženou politiku a tag „domů“ spustí jinou sekvenci. Hugging Face kredit může pomoci s tréninkem, ale výsledek stále závisí na správném modelu simulace, reward funkci a testování na skutečném robotu.

Senzory a odhad nákladů

To, čemu Pollen říká „LiDAR“, je ve skutečnosti 8×8 ToF matice: malý čip za pár stovek korun, ne rotující lidar za tisíce. Právě proto se do robota za 399 USD vejde „kamera i LiDAR“.

Komponenta	Co to je	Odhad ceny v kusovce
„LiDAR“	8×8 ToF matice, prakticky jistě ST VL53L5CX / VL53L8CX (dosah ~4 m, 64 zón, I ² C)	250–400 Kč
Kamera	širokoúhlá MIPI-CSI kamera třídy OV5647 / IMX219	150–400 Kč
2× IMU	6osé MEMS (BMI270, ICM-42688, LSM6DSO)	50–150 Kč/ks
Výpočet	Rockchip RK3566 s NPU, 1 GB RAM, 32 GB flash (deska třídy Radxa Zero 3)	800–1 200 Kč
15× servo	sériové sběrníkové servo (Open Duck Mini používal Feetech STS3215)	300–450 Kč/ks
Baterie	NP-F550 (kamerový standard Sony), ~1 h provozu	250–400 Kč

Senzory dohromady stojí pod 1 000 Kč. Cena robota je hlavně v servech (~5 000 Kč) a mechanice.

Co ToF matice umí a neumí

- **Umí:** detekovat překážku před sebou, odhadnout vzdálenost k míči nebo zdi, poznat okraj stolu (spodní zóny hlásí „nekonečno“), zhruba určit směr k nejbližšímu objektu.
- **Neumí:** mapovat místnost jako SLAM lidar. 64 pixelů hloubky, žádná rotace, žádný 360° sken. Je to hloubková kamera s velmi malým rozlišením.

Kamera na RK3566 s NPU zvládne lehké modely (detekce míče, tváře, čtení ArUco značek). Proto RK3566 a ne Raspberry Pi Zero.

Co je reálné pro Duckbot

Vidění, fáze 1 (stačí ESP32):

- **VL53L5CX modul** (Pololu, Adafruit, klony) ≈ 300 Kč. Připojí se přes I²C rovnou na ESP32, dává 64 vzdáleností při 15 Hz. Detekce překážky a míče před zobákem.
- Levnější alternativa **VL53L1X** (jedna zóna, ~150 Kč): jen „něco je přede mnou ve vzdálenosti X“.

Vidění, fáze 2 (potřebuje větší počítač):

- **ESP32-S3 + OV2640** (ESP32-CAM, 200–300 Kč): streamování obrazu a velmi jednoduchá detekce barvy (oranžový míč). Na neuronové síti je slabý.

- **Raspberry Pi Zero 2 W + Camera Module 3** (~1 500 Kč) nebo **Radxa Zero 3W** (~1 100 Kč, stejný RK3566 jako Microduck): tady už poběží ONNX politika i detekce objektů, ESP32 zůstane jako realtime vrstva pro serva.

IMU: BMI270 nebo ICM-42688 modul za 100–150 Kč, na první pokusy MPU-6050 za 50 Kč (horší drift).

Čemu se vyhnout: rotační lidary (RPLidar A1 ~2 500 Kč, LD19 ~1 500 Kč) jsou na 800g robota těžké, žerou proud a pro chůzi nic nepřinesou. Hloubkové kamery RealSense / OAK-D (5 000+ Kč) nedávají smysl, dokud robot nestojí.

Doporučený nákup na začátek: ESP32-S3, BMI270, VL53L5CX, celkem pod 700 Kč. To je stejná senzorická sada jako Microduck bez kamery. Kameru a RK3566 / Pi přidat, až chůze funguje.

Kam se podívat po simulátoru

1. Microduck simulátor: Hugging Face Microduck Simulator

Začal bych zde. Nejdřív si vyzkoušej model, scénu, ovládání a chování po pádu. Zapiš si, jaké vstupy simulátor používá a jaké akce vrací. Tyto rozměry budou později určovat firmware i síť pro Duckbot.

2. Oficiální runtime a dokumentace: pollen-robotics/microduck

Projdi hlavně README, docs/design/architecture.md, docs/design/robotd-design.md, docs/robot/cheatsheet.md a docs/policy-manifest.md.

3. Sim2real a RL: pollen-robotics/microduck_rl

Tréninková prostředí (mjlal), export politiky do ONNX a nástroj publish pro nahrání na Hugging Face Hub. Repozitář je od září 2026 veřejný.

4. MuJoCo: MuJoCo documentation

To je fyzikální simulátor. Nejprve stačí umět načíst model, spustit scénu, číst stav kloubů a IMU a posílat cíle do serv. PPO a vlastní trénink přijdou až po ověření, že model odpovídá skutečné mechanice.

Jak se politika dostává do robota

Robot nespouští simulaci, spouští **politiku**: neuronovou síť s pevným rozhraním. Simulace (MuJoCo) je jen trenažér, kde se síť učí.

```
vstup (observation):  úhly a rychlosti serv, IMU (gyro, gravitace),  
                      poslední akce, požadovaná rychlost (x, y, otočení)  
                      |  
                      ONNX model (50 Hz)  
                      |  
výstup (action):      cílové úhly pro serva
```

U Microducku má graf tvar [1, 61] -> [1, 14]: 61 čísel pozorování dovnitř, 14 cílových úhlů ven.

Co je ONNX

ONNX = **Open Neural Network Exchange**. Je to otevřený formát souboru pro uložení natrénované neuronové sítě nezávisle na frameworku. Síť se natrénuje v PyTorch, exportuje se do policy.onnx (jednotky MB) a na robota ji spouští **ONNX Runtime**, malá knihovna, která nepotřebuje PyTorch ani GPU. Proto to běží na RK3566 s 1 GB RAM.

Cesta souboru

```
MuJoCo / mjlal + PPO trénink (PC s GPU)      repozitář microduck_rl  
| export  
policy.onnx + manifest.json  
| uv run publish  
Hugging Face Hub (jeden .onnx na repozitář)  
| robotctl policy search / load  
RK3566 na palubě robota, ONNX Runtime, smyčka 50 Hz  
|  
serva
```

Robot je linuxový počítač na Wi-Fi. Politiky se do něj nekopírují ručně, ale přes nástroj `robotctl policy list | load | reset | check | update | search`, který je stahuje z Hugging Face Hubu.

Kde ONNX soubory sehnat

- **Oficiální politiky:** `pollen-robotics/microduck-policies`. Pollen tam 31. srpna 2026 zveřejnil devět politik, které se dodávají s robotem (chůze, sed, kop, zvednutí po pádu atd.).
- **Komunitní politiky:** na Hubu vznikají další, například `q2p/microduck-beak-throw` nebo `HannesVonEssen/microduck-running`. Přehled udržuje `awesome-microduck`.
- **Vlastní trénink:** `pollen-robotics/microduck_rl` obsahuje tréninková prostředí (mjlab), export do ONNX a příkaz `publish`, který před nahráním zkontroluje tvar grafu, otestuje výstup na vzorových vstupech a doplní `manifest.json` podle schématu z `docs/policy-manifest.md`.

Proč to na skutečném robotu funguje (sim2real)

Síť se učila v simulaci s ideálními servy a podlahou. Aby fungovala i na hardwaru:

- **MJCF model musí sedět:** hmotnosti, délky, rozsahy kloubů, rychlost a moment serv.
- **Domain randomization:** při tréninku se náhodně mění tření, zpoždění serv, hmotnosti a šum IMU, síť se naučí snést, že realita je trochu jiná.
- **Stejné rozhraní:** pořadí serv, jednotky (radiány) a frekvence smyčky musí být na robotu identické se simulací. Jedna prohozená noha a robot spadne.

Co z toho plyne pro Duckbot

Stažená politika Microducku na Duckbotu fungovat nebude: je natrénovaná na jinou mechaniku, jiná serva a 61/14 rozhraní. Použitelný je postup, ne soubor. ONNX inference na ESP32 rozumně nepoběží, takže:

```
PC / Raspberry Pi      ESP32
ONNX politika, 50 Hz -> přijme cílové úhly
(Wi-Fi / USB seriál)   omezí rozsah a rychlost
                        <- pošle stav serv + IMU
```

První reálný krok ale není simulace: nejdřív ručně naprogramované pózy a stoj z IMU. Simulace má smysl, až je hotová mechanika a změřené parametry, jinak trénujeme politiku pro robota, který neexistuje.

Jak bych programoval Duckbot

ESP32: bezpečný realtime základ

ESP32 by měl vlastnit nízkourovňové řízení:

- čtení IMU,
- řízení servomotorů,
- kalibraci nulových poloh,
- omezení úhlů a rychlosti,
- detekci pádu,
- nouzové vypnutí serv,
- watchdog a timeout příkazů.

ESP32 nesmí čekat na Wi-Fi, hlasový model ani pomalý server. Když nepříjde příkaz například 200 ms, musí přejít do bezpečného režimu a zastavit pohyb.

Pro první pokusy stačí MPU6050. Pro stabilnější měření bych později použil BNO085 nebo podobnou IMU s vlastní fúzí senzorů. Akcelerometr sám o sobě při chůzi nestačí: dynamika pohybu způsobuje zrychlení, které se plete s gravitací.

Raspberry Pi nebo jiný SBC: mozek a komunikace

Raspberry Pi 5, případně podobný Linuxový počítač, by řešil:

- MuJoCo a testovací nástroje,
- inference ONNX politiky,
- kameru a rozpoznávání objektů,
- hlasové rozpoznávání,
- převod vysokých příkazů na bezpečné záměry,
- logování a aktualizace.

ESP32 a SBC mohou komunikovat přes USB sériovou linku, UART nebo CAN. Pro první verzi je nejjednodušší USB serial s řádky JSON. Až bude protokol stabilní, lze přejít na binární rámce nebo CAN.

Příklad příkazu, který má smysl posílat:

```
{"type": "intent", "mode": "stand", "forward": 0.0, "turn": 0.0, "ttl_ms": 200}
```

Příklad příkazu, který bych neposílal z hlasového asistenta ani z Wi-Fi:

```
{"servo": 7, "angle": 1.42}
```

Přímé úhly musí zůstat pod kontrolou bezpečnostní vrstvy na ESP32 nebo v lokálním řídicím procesu.

Co bude Duckbot umět v jednotlivých etapách

Duckbot v0.1: stolní prototyp

- číst náklon a teplotu IMU,
- řídit dva až čtyři servomotory,
- nastavit bezpečnou nulovou polohu,
- reagovat na tlačítko a nouzové vypnutí,
- posílat telemetrii do počítače.

Tady ještě netrénujeme chůzi. Cílem je ověřit napájení, mechaniku, servo limity a komunikaci.

Duckbot v0.2: stabilní stoj

- vícekloubové nohy,
- stoj podle předem připravené pózy,
- kompenzace náklonu pomocí jednoduchého PID regulátoru,
- detekce pádu,
- bezpečné uvolnění nebo vypnutí motorů.

Toto je nejdůležitější milník. Když robot bezpečně nestojí, reinforcement learning pouze urychlí ničení serv a převodů.

Duckbot v0.3: předprogramovaná chůze

- krok dopředu a dozadu,
- otočení na místě,
- sednutí a postavení,
- ovládání gamepadem,
- záznam kloubů, IMU a příkazů.

Nejdřív použijeme trajectory-based řízení: ručně navržené trajektorie kloubů, interpolaci a stabilizační korekci. To je levnější a lépe laditelné než okamžité trénování neurální sítě.

Duckbot v0.4: simulace a sim2real

- přesný MJCF model v MuJoCo,
- stejné názvy kloubů a stejné pořadí akcí jako ve firmware,
- náhodné tření, hmotnost, zpoždění serv, síla motorů a šum senzorů,
- trénink PPO v simulaci,
- export politiky do ONNX,
- opatrné testování na zavěšeném nebo jištěném robotu.

Teprve tady se dostáváme k principu Microducku: politika neříká „nastav servo na přesný úhel“, ale z pozorovaného stavu vypočítá další akci. Pozorování může obsahovat úhly a rychlosti kloubů, orientaci z IMU, rychlost těla a požadovaný směr chůze.

Co můžeš robota naučit

Nejprve klasickým programováním

- stát a sedět,
- reagovat na tlačítka,
- otočit hlavu za zvukem nebo světlem,
- sledovat barevný míček,
- přijet nebo přijít k cíli,
- kvákat podle události,
- bezpečně se zastavit při pádu nebo slabé baterii.

Později učení v simulaci

- stabilní chůzi,
- sledování rychlosti a směru,
- překonání mírně jiné podlahy,
- kopnutí do míče,
- vstávání z vybraných pádových poloh,
- chování při změně hmotnosti nebo tření.

Hlasový asistent

Hlasový asistent by měl být vysoká vrstva, ne stabilizační smyčka. Příkaz „Duckbote, pojd' dopředu“ se převede na `forward=0.2`, zatímco ESP32 stále hlídá náklon, limity a timeout. Příkaz „pojd' za mnou“ bude potřebovat kameru nebo vzdálenostní senzor a vlastní lokální řízení; samotný LLM neumí bezpečně řídit serva v realtime.

První hlasové příkazy mohou být:

- „Stůj“,
- „sedni“,
- „postav se“,
- „jdi dopředu“,
- „otoč se doleva“,
- „kvákní“.

Doporučené pořadí práce

1. Spustit Microduck simulátor a pochopit pozorování, akce a reset epizody.
2. V Duckbotu rozchodit IMU, jedno servo a telemetrii.
3. Přidat bezpečnostní limity, watchdog a nouzové vypnutí.
4. Postavit stojící mechanismus a změřit skutečné úhly, vůle a odběr.
5. Napsat ruční stoj a jednoduchý krok.
6. Teprve potom vytvořit MJCF model vlastní mechaniky.
7. Porovnat simulovanou a skutečnou odezvu serv.
8. Trénovat nejprve stoj, potom chůzi, nakonec triky jako kopnutí nebo vstávání.
9. Hlas a kameru přidat až nad stabilní pohybový základ.

Realistické rozhodnutí

Vlastní Duckbot může napodobit hlavní myšlenku Microducku, ale ne jeho hotové schopnosti za cenu několika ESP32 součástek. Nejtěžší část bude mechanika, serva, napájení, kalibrace a sim2real. Výhoda vlastního projektu je, že každou vrstvu pochopíš a můžeš ji měnit.

Microduck bych proto bral jako referenční implementaci a testovací etalon. Duckbot bych stavěl tak, aby první funkční verze nevyžadovala neurální síť. Když bude v0.3 bezpečně chodit s ruční trajektorií, bude mít smysl investovat čas do MuJoCo a PPO.

Odkazy

- [Microduck: oficiální repozitář](#)
- [Microduck: oficiální produktová stránka](#)
- [Microduck Simulator na Hugging Face](#)
- [Microduck RL: tréninková prostředí](#)
- [Microduck policies na Hugging Face](#)
- [awesome-microduck: přehled komunitních politik a nástrojů](#)
- [Microduck press kit: specifikace](#)
- [MuJoCo dokumentace](#)
- [TorchRL PPO dokumentace](#)